A

MINI PROJECT REPORT ON

Topic Name- Smart Traffic Signal Using Q-Learning

SUBMITTED TO SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF DEGREE OF

BACHELOR OF TECHNOLOGY (B.TECH)

(Artificial Intelligence & Machine Learning)

BY

Student Name	Roll no
Yash Sachin Bonde	32251
Sakshi Sunil Zunzunwala	32252
Aishwarya Pramod Patil	32253
Vicky Harishkumar Badmera	32254
Divya Dilip Tirodkar	32255

Under The Guidance Of

Mrs. Preeti Gaikwad

Dr. D. Y. Patil College of Engineering and Innovation, Varale

Department of Artificial Intelligence - Data Science and Machine Learning

Dr. D.Y. Patil College of Engineering & Innovation
Varale – Talegaon, Pune – 410507

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

Academic Year: 2026 – 2027

Dr. D. Y. Patil College of Engineering and Innovation, Varale

Department of Artificial Intelligence - Data Science and Machine Learning

CERTIFICATE

This is to certify that the Mini Project entitled

"Smart Traffic Signal Using Q-Learning"

Student Name	Roll no
Yash Sachin Bonde	32251
Sakshi Sunil Zunzunwala	32252
Aishwarya Pramod Patil	32253
Vicky Harishkumar Badmera	32254
Divya Dilip Tirodkar	32255

is a bonafide work carried out by them under the guidance of Mrs. Preeti Gaikwad, in partial fulfillment of the requirements of Savitribai Phule Pune University, Pune for the award of the degree of Bachelor of Technology in Artificial Intelligence and Data Science.

Mrs. Preeti Gaikwad
Guide

Dr. Dipannita Mondal
Head- AI- DS & ML

Dr. Suresh Mali
Director

Signature of Internal Examiner

Signature of External Examiner

ACKNOWLEDGENT

It is our pleasure to acknowledge sense of gratitude to all those who helped us in completion of Mini project. I am highly indebted to **Mrs . Preeti Gaikwad** for their guidance and constant supervision as well as for providing necessary information regarding the internship & also for their support. I would like to express my gratitude towards Head of Artificial Intelligence - Data Science and Machine Learning Department **Dr. Dipannita Mondal** for their kind co-operation and encouragement which help me for providing the required facilities. Finally, we wish to thank and appreciate to all our teachers and friends for their constructive comments, suggestions and guidance and all those directly or indirectly helped us in completing this Mini Project.

Name of the Student and Sign

Place:

Date:

Contents

1	Chapter 1: Introduction	**5**
2	Chapter 2: Problem Statement	6
3	Chapter 3: Objectives	8
4	Chapter 4: Motivation	9
5	Chapter 5: Scope and Rationale of the Project	11
6	Chapter 6: Methodological Details	12
7	Chapter 7 : Results	15
8	Chapter 8: Analysis	17
9	Chapter 9: Inferences	18
10	Chapter 10: Conclusion	19
11	Chapter 11: Suggestions	20
12	Chapter 12: Acknowledgments	21
13	Chapter 13 : List of Refernces	22

Chapter 1: Introduction

Traffic congestion is a common problem at road intersections. Fixed-time traffic signals allocate the same green-light duration to each road even when traffic density changes. This mini project presents a small Python simulation of a two-direction intersection and applies Q-learning, a reinforcement learning technique, to select signal actions based on observed traffic queues.

- Model North–South and East–West traffic at a single intersection.
- Represent traffic conditions using states derived from queue lengths.
- Use Q-learning to select a signal action and update a Q-table from rewards.
- Compare the learning-based controller with a fixed-time signal using the same vehicle-arrival inputs.

Background: Traffic signals are often controlled by preset timings. Such timings are easy to implement but may not respond efficiently to changing traffic demand.

Reinforcement Learning: Q-learning is a model-free method that learns action values through repeated interaction with an environment.
Technical Primer: The simulation defines states, actions, rewards, learning rate, discount factor, and an exploration strategy. The Q-table stores estimated values for state–action pairs.
Project Approach: A lightweight, reproducible simulation is used so that the behaviour of fixed-time and learning-based signal control can be observed without connecting to real traffic infrastructure.

Chapter 2: Problem Statement

At a road intersection, traffic volume can vary from one direction to another. A fixed-time controller may assign green time to an approach with a short queue while vehicles accumulate on the other approach. The project investigates whether a simple Q-learning agent can make signal decisions based on the simulated queue state.

- Fixed signal timings do not directly respond to the current queue lengths.
- Unequal vehicle arrivals can create unnecessary waiting and longer queues in one direction.
- A learning agent requires a clear definition of states, actions, rewards, and safe signal transitions.

This project aims to:

1. Build a Python simulation for North–South and East–West traffic queues.
2. Implement a Q-learning controller that selects which direction receives green time.
3. Compare the learning-based controller with a fixed-time baseline under identical arrival conditions.
4. **Industry-Specific Risks**:
    1. **Healthcare**: 28% of patient portals in developing nations use HTTP, risking HIPAA violations (HIPAA Journal, 2023).
    2. **E-Commerce**: 15% of SMEs skip HTTPS due to perceived costs, leaving payment gateways vulnerable.

5. **Technical Barriers**: Certificate expiration mismanagement causes 12% of HTTPS sites to revert to HTTP (Let's Encrypt, 2023).

User Awareness: Only 32% of users recognize the padlock icon's significance (Pew Research, 2023)

Chapter 3: Objectives

1. Demonstrate HTTP Vulnerabilities:

- Represent queue lengths as a compact traffic state.
- Define actions for granting green time to North–South or East–West traffic.

2. Implement HTTPS Protections:

- Create and update a Q-table using the Q-learning update rule.
- Use an exploration strategy during training and greedy action selection during evaluation.

3. Compare Security Postures:

- Measure queue lengths and vehicles served.
- Run both controllers with identical vehicle-arrival sequences for a fair comparison.

4. Expanded Technical Goals:

- Keep the simulation small enough to understand and demonstrate in a mini-project.
- Use standard Python so the core simulation does not depend on external libraries.

5. Educational Outreach:

- Develop a workshop for local businesses on migrating to HTTPS.
- Create a GitHub repository with step-by-step SSL configuration guides.

Chapter 4: Motivation

The motivation for this project is to explore how an adaptive controller can make traffic-signal decisions from the current traffic situation rather than relying only on preset timings.

1. **Escalating Threats**:
    - Traffic demand changes over time, and one fixed timing plan may not suit every traffic condition.
    - When one direction has a longer queue, an adaptive signal may be able to allocate green time more appropriately.
    - Q-learning provides a practical introduction to reinforcement learning because its state, action, reward, and Q-table can be inspected.
    - A small simulation allows the team to test ideas safely without controlling real traffic lights or collecting road-side data.
2. **Educational Gap**:
    - The project demonstrates the difference between a rule-based baseline and a controller that improves its action-value estimates through repeated experience.
    - The results can be explained using visible queue lengths, selected signal phases, and a comparison of summary measures.
3. **Regulatory Pressures**:
    - The project connects Python programming, data structures, decision-making, and introductory reinforcement learning in one practical example.
    - The implementation is intended for learning and simulation only; it does not replace certified traffic-control systems.

1. System Architecture:

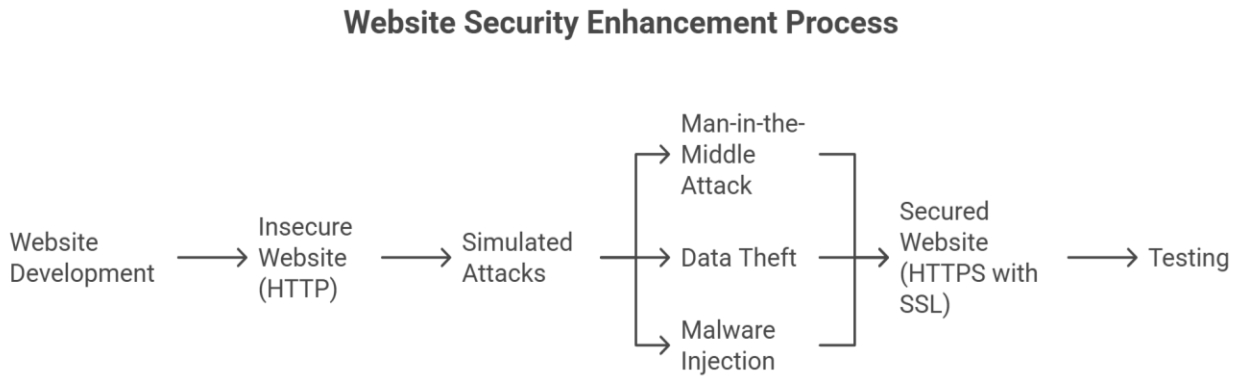


Image Description:

The system generates vehicle arrivals, observes the North–South and East–West queues, forms a state, selects a signal action, applies a reward, and updates the Q-table during training. During evaluation, the learned policy and a fixed-time controller are tested with the same arrival sequence and their queue-related measures are compared.

Chapter 5: Scope and Rationale of the Project

Scope:

- **Technical Boundaries**:
    - Focuses on one simulated intersection with two traffic approaches: North–South and East–West.
    - Includes vehicle arrivals, queue lengths, signal phases, a Q-table, reward calculation, and a fixed-time baseline.
- **Practical Constraints**:
    - Uses a local Python simulation with generated vehicle arrivals; no real traffic signal hardware is connected.
    - Does not include multiple intersections, pedestrian modelling, emergency-vehicle priority, camera-based vehicle detection, or deployment on public roads.

Rationale:

1. **Hands-On Learning**:
    - The queue state and selected signal action can be displayed step by step, making the reinforcement-learning process easier to understand.
2. **Cost-Effective Advocacy**:
    - The simulation can be developed using standard Python and run on a typical student laptop.
3. **Microcosm of Real-World Impact**:
    - The same traffic-arrival sequence can be replayed for both controllers, helping make the comparison more meaningful.

Chapter 6: Methodological Details

The project methodology is divided into the following phases:

1. Traffic Environment Development

- **Technology Used:** Python and its standard library.
- **Environment Features:**
    - Maintain separate vehicle queues for North–South and East–West approaches.
    - Generate vehicle arrivals at each time step using configurable probabilities or a predefined sequence.
    - Track the active signal phase and allow only the selected approach to discharge vehicles during green time.

2. State, Action, and Reward Design

- **State:** A compact representation of the two queue lengths, optionally grouped into ranges to keep the Q-table small.
- **Actions:**
    - Give green time to North–South traffic.
    - Give green time to East–West traffic.
    - Use a controlled phase-change rule so the simulation avoids unrealistic instantaneous switching.

3. Q-Learning Controller

- **Q-Table:**
    - Store estimated values for each state–action pair using a Python dictionary or a two-dimensional structure.
    - Learning Steps:
        1. Observe the current state of both queues.
        2. Choose an action using epsilon-greedy exploration during training.
        3. Apply the action, advance the simulation, and observe the next state.
    - Calculate a reward that favours lower total queues or reduced waiting and penalises excessive congestion.

- **Q-Value Update:**
    - Use the standard Q-learning update rule with learning rate alpha and discount factor gamma.
    - Training and Evaluation:
        1. Repeat episodes to update the Q-table.
        2. Evaluate the learned policy separately, with exploration reduced or disabled.
    - Record queue-related measures for each controller.

4. Baseline Comparison

- Fixed-Time Controller:
- Alternate green time between the two approaches using a preset duration.

Comparison Procedure:

- Run the fixed-time and Q-learning controllers with identical vehicle-arrival inputs and comparable simulation duration.
    1. Record average or total queue length, vehicles served, and waiting-related indicators.

2. Compare the summaries and inspect sample signal decisions.
3. Report any limitations, including the simplicity of the simulated traffic model.

- **Testing**:
    - Check that the Q-table changes during training and that the controller always selects a valid signal action.
    - Repeat evaluation with the same arrival sequence for both controllers; report measured values from actual runs rather than assuming the learning controller will always perform better.

Chapter 7: Results

Aspect	Fixed-Time Controller	Q-Learning Controller
Signal Decision	Preset alternating green durations	Selects action from current queue state
Traffic Awareness	Does not adapt to queue lengths	Uses the represented queue state
Learning	No training required	Updates Q-values from rewards
Evaluation	Baseline for comparison	Evaluated after training
Queue Length	Record measured value	Record measured value
Vehicles Served	Record measured value	Record measured value
Waiting Indicator	Record measured value	Record measured value

Key Observations:

- The simulation should display the active signal phase, vehicle arrivals, queue lengths, selected action, and reward at each step.
- A comparison table should report the measured results for fixed-time and Q-learning control using identical traffic-arrival inputs.

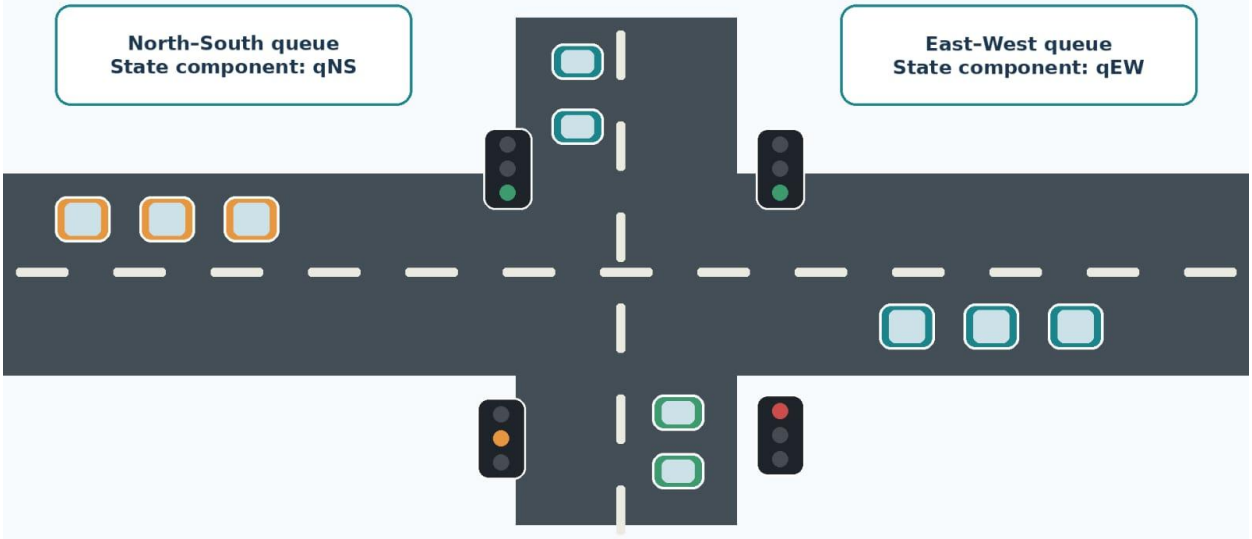
EXAMPLE Q-TABLE STRUCTURE

Traffic State	Action: N–S Green	Action: E–W Green
Short N–S / Long E–W	$Q(s, a_1) = 0.8$	$Q(s, a_2) = 1.7$
Long N–S / Short E–W	$Q(s, a_1) = 2.1$	$Q(s, a_2) = 0.6$
Both queues balanced	$Q(s, a_1) = 1.2$	$Q(s, a_2) = 1.1$

Illustrative values only — actual Q-values are learned during training.

Choose the action with the higher Q-value during evaluation.

SIMULATED TWO-DIRECTION INTERSECTION



Illustrative simulation only — not connected to real traffic signals.

Chapter 8: Analysis

Key Findings:

- The fixed-time controller is a simple baseline and does not use current queue information to alter its preset schedule.

- The Q-learning controller estimates the long-term value of actions using rewards and the next state's maximum Q-value.
    - Queue-based rewards encourage the agent to reduce congestion in the simulated environment.
    - The quality of learning depends on state design, reward design, exploration, training episodes, and traffic-arrival patterns.

- Using the same arrival sequence for both controllers reduces the effect of random input differences on the comparison.

- **Evaluation Measures**:
    - Compare average queue length and total vehicles served over the same number of simulation steps.
    - Also inspect waiting-related measures and the stability of decisions after training; include actual measured values in the final submission.

- **Limitations:**
    - The simulation simplifies real traffic: it does not model road geometry, turning movements, pedestrians, lane changes, or real-world driver behaviour.
    - Results from a small simulation cannot establish that the controller is ready for deployment at a real intersection.

- A small Q-table may be easier to understand but can lose detail when queue lengths are grouped into broad state ranges.

Chapter 9: Inferences

- **Implementation Responsibility:**
  Keep the random seed or arrival sequence fixed when comparing controllers so that the experiment can be repeated.
  Separate training from evaluation to avoid presenting exploration behaviour as the final policy.

- **Reward Design:** Poorly selected rewards may favour short-term queue reduction while ignoring fairness between approaches.
  .
- **Evidence:** Claims about improvement should be based on recorded runs and reported metrics, not assumed in advance.

- Q-learning can adapt its policy to simulated traffic patterns, but performance depends on the environment and training configuration.

- A baseline comparison is essential because a learning-based method is not automatically better than a simple controller.

Chapter 10: Conclusion

This project outlines a small reinforcement-learning simulation for adaptive traffic-signal control using Python.

- It models two traffic directions and maintains their vehicle queues.
- It uses Q-learning states, actions, rewards, and Q-table updates to learn signal decisions.
- It compares the learned policy with a fixed-time controller under identical vehicle-arrival inputs.

The project provides a manageable demonstration of how reinforcement learning can be applied to a practical decision-making problem.

- **Future Trends**:
  - Future work could include more realistic arrival patterns, minimum green times, yellow/all-red transition phases, and pedestrian safety rules.
  - Further extensions could use multiple intersections or a traffic simulator, provided the scope and validation requirements are increased.
- A future version could visualise queue lengths and compare results across several traffic-demand scenarios.

Chapter 11: Suggestions

1. For Developers:

- Keep state and action definitions small and clearly documented.
- Use a reward that reflects queue reduction while avoiding starvation of either approach.
- Separate training, testing, and evaluation code.
- Use fixed random seeds or stored arrival sequences for reproducible comparisons.
- Record actual measurements and explain the experiment settings.

2. For Project Evaluation:

- Demonstrate the simulation step by step and explain the current state and chosen action.
- Show how Q-values change during training and how the learned policy behaves during evaluation.
- Compare both controllers using identical vehicle arrivals and equal simulation duration.
- Discuss limitations rather than claiming guaranteed real-world traffic improvements.

3. For Future Improvements:

- Add pedestrian crossings and safe phase-transition timing.
- Test low, medium, and high traffic-demand scenarios.
- Explore a richer state representation if the current Q-table is too coarse.
- Consider a traffic simulator only after the basic Python model is working.

Chapter 12: Acknowledgments

I sincerely thank **Dr. Dipannita Mondal**, HOD of AI –DS & Ml, for her guidance, and **Mrs. Preeti Gaikwad** for her expert mentorship in web security. My gratitude extends to our department for providing an environment that supports practical learning.

I deeply appreciate my project team – Yash Bonde,Sakshi Zunzunwala,Aishwarya Patil,Vicky Badmera and Divya Tirodkar. We thank our team members for their collaboration in designing the traffic environment, implementing the Q-learning logic, testing the fixed-time baseline, and reviewing the report. Student names and roll numbers should be added to the cover page before submission

We also acknowledge the Python documentation and educational resources on reinforcement learning and Q-learning that support the technical understanding behind this project.

Chapter 13: List of References

- Python Documentation: https://docs.python.org/3/
- Sutton, R. S., & Barto, A. G. Reinforcement Learning: An Introduction (2nd edition): http://incompleteideas.net/book/the-book-2nd.html
- Watkins, C. J. C. H. (1989). Learning from Delayed Rewards. PhD thesis, University of Cambridge.
- OpenAI Gymnasium Documentation (environment concepts): https://gymnasium.farama.org/
- GeeksforGeeks, Q-learning in Reinforcement Learning: https://www.geeksforgeeks.org/machine-learning/q-learning-in-reinforcement-learning/